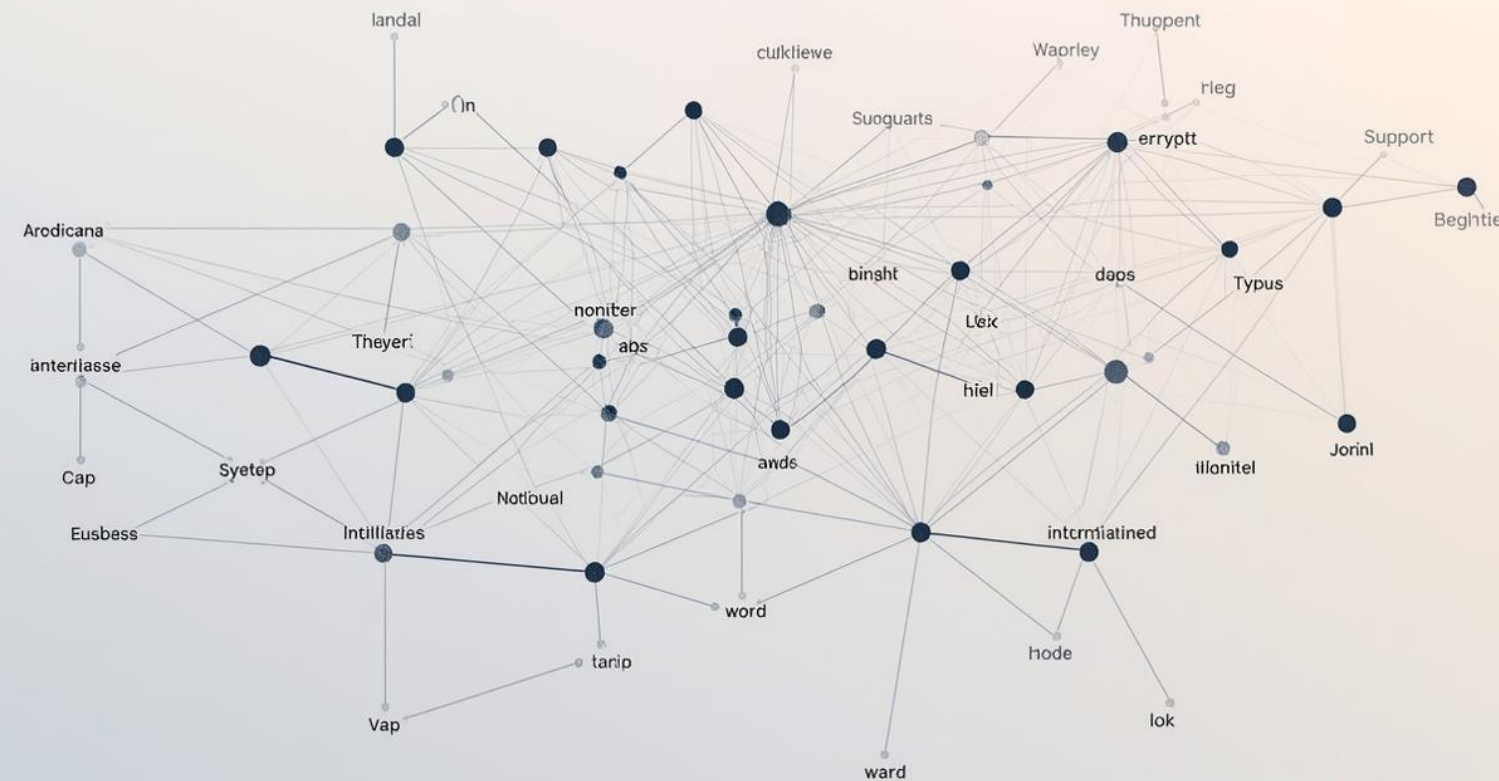


Word Vectorization



Introduction to Word Vectorization

- Converting Words/sentences into linearly independent vectors.
- Representing the words in high-dimensional space.
- Individual vectors represent the properties of Words/Sentences.
- The process of converting words into vectors is also called word embeddings.
- Vector operations are the core idea behind neural network training.
- Are word embeddings and text vectorisation the same???

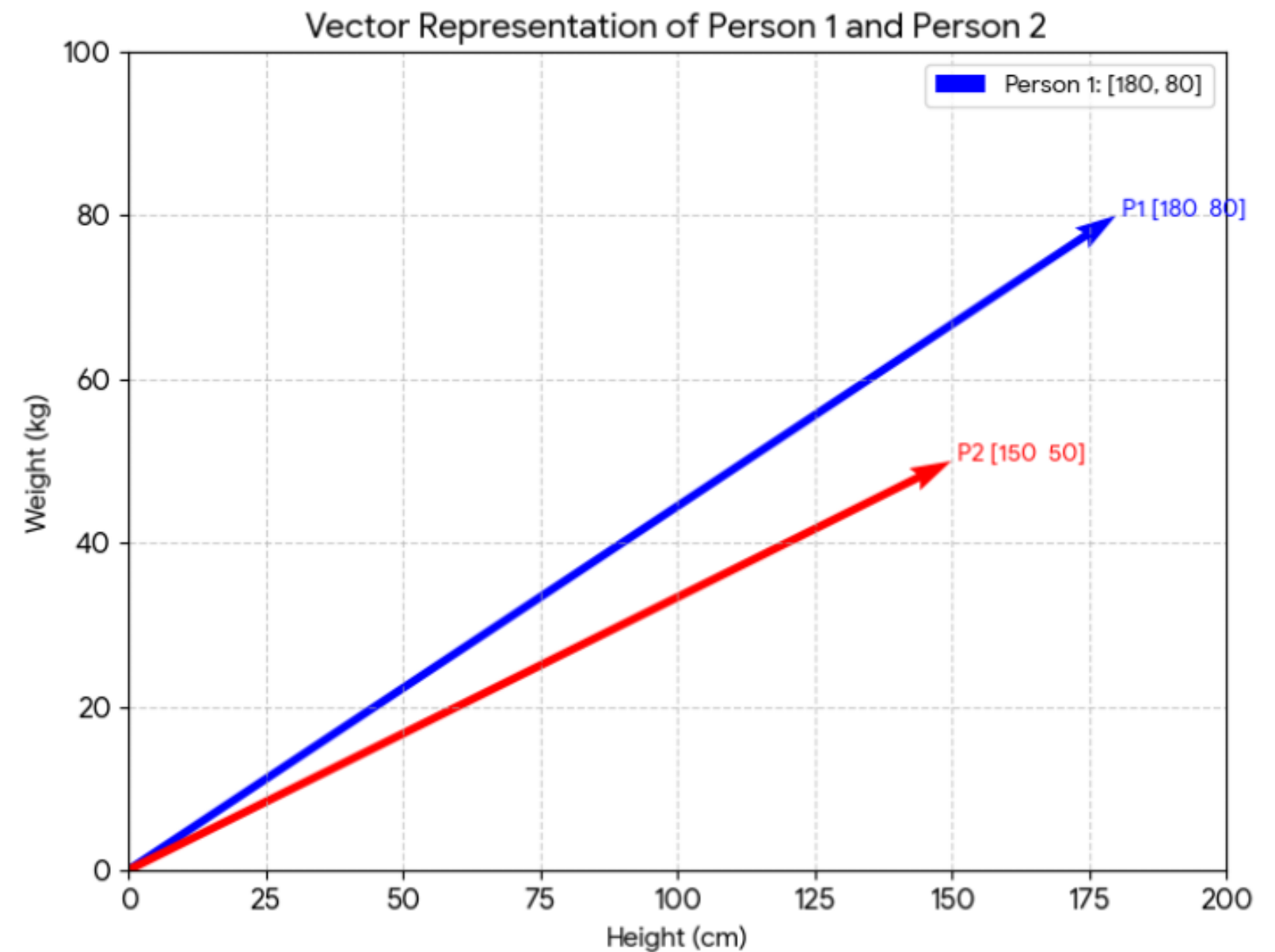


Importance of Word Vectorization

Let's define two people using three traits: Height (cm), Weight (kg), and Age (years).

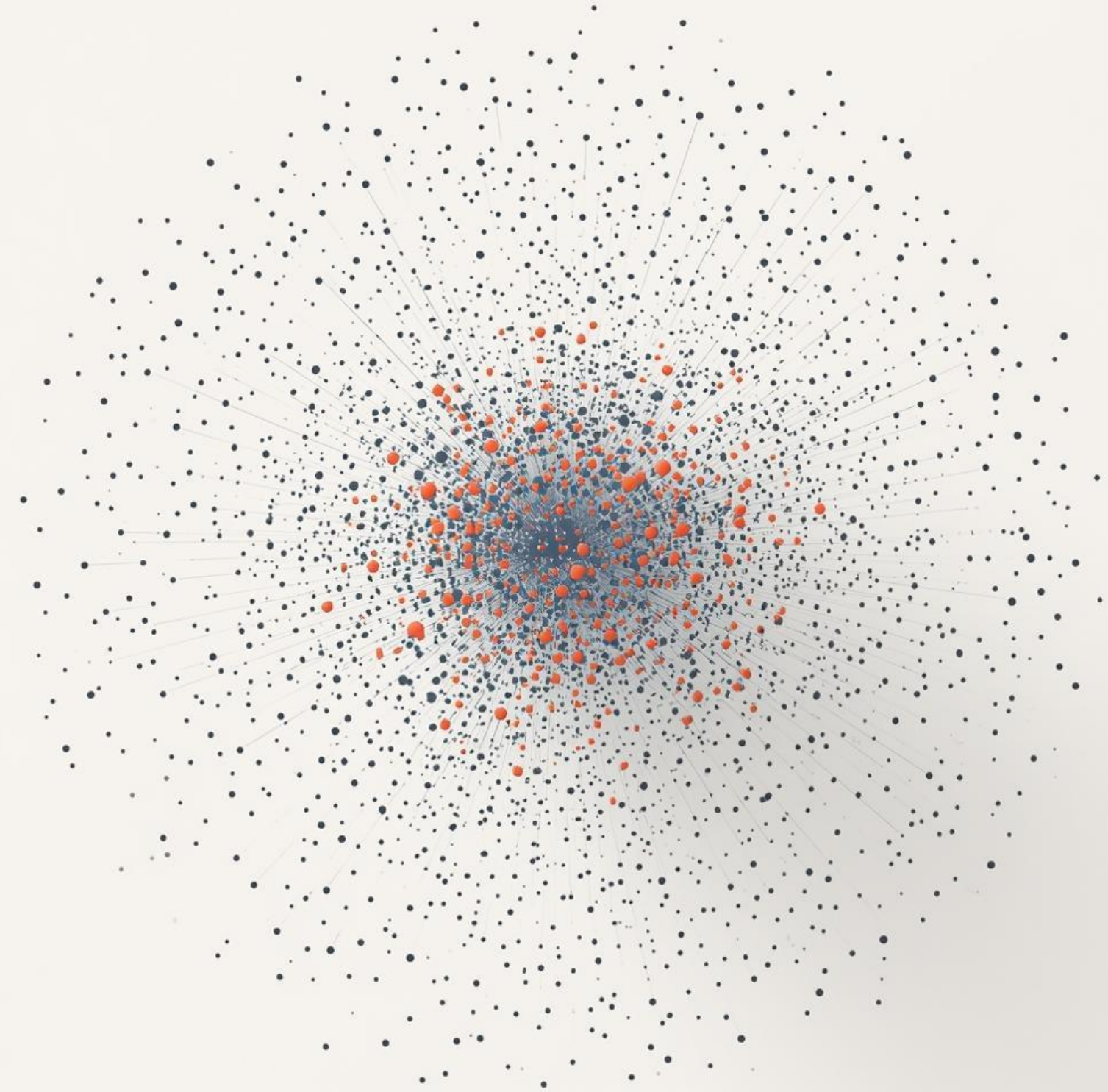
Person 1: [180, 80, 25]

Person 2 : [150, 50, 45]



Embeddings representation in N-dimensional space

We can use t-SNE plot to visualise embeddings



Statistical and Count-Based Methods

Exploring Key Differences Between Statistical/Count-Based and Predictive/Neural Approaches

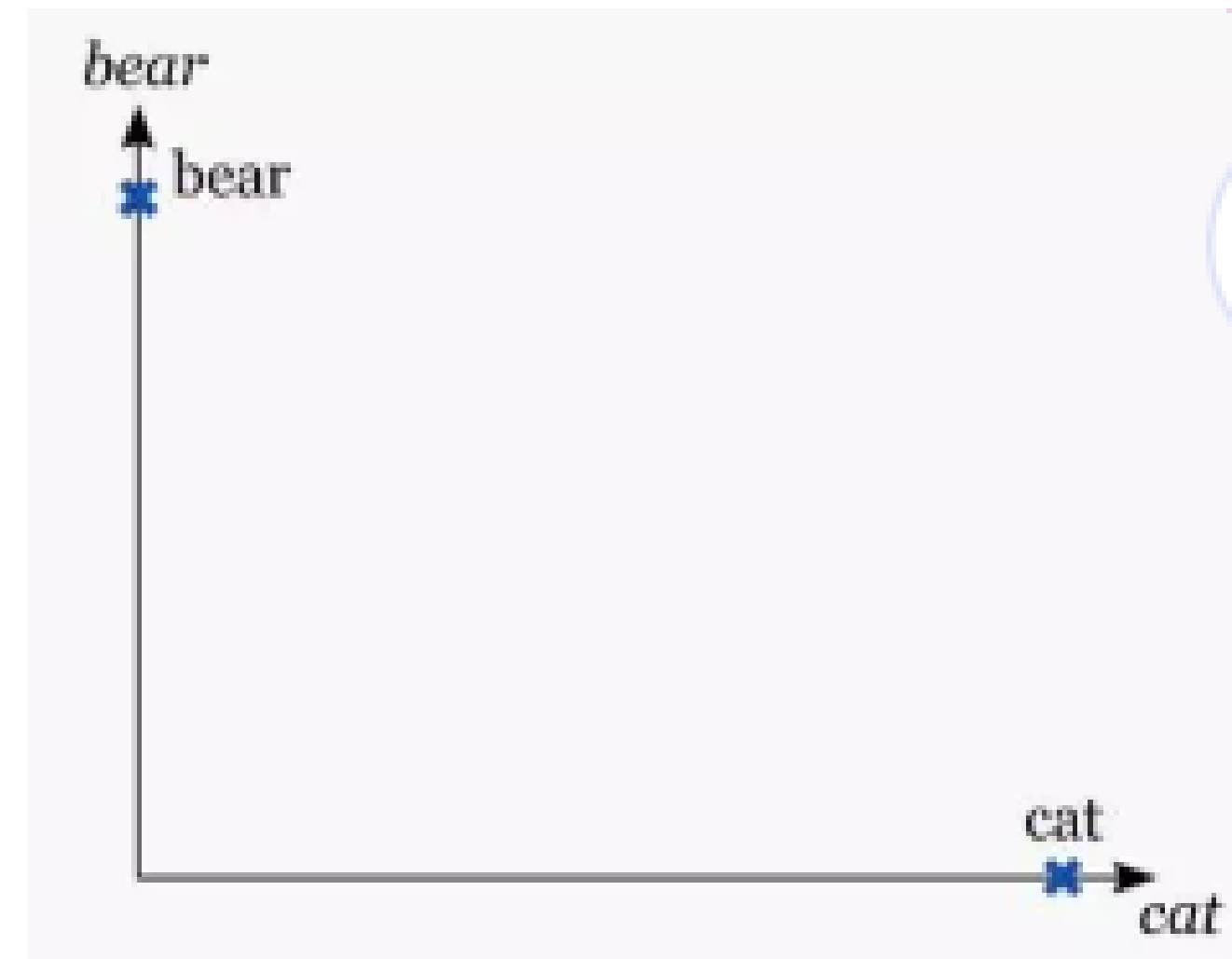
Statistical and count-based vectorization methods serve as foundational techniques in natural language processing (NLP). They focus on quantifying word occurrences and relationships to form numerical representations, contrasting with predictive and neural methods that leverage complex models for understanding context and meaning. Understanding these differences is crucial for advancing NLP applications.



One-Hot Encoding

Understanding Binary Vector Representation for Natural Language Processing

One-hot encoding is a technique used to represent words as binary vectors. Each word is mapped to a unique position in a vector, where all values are zero except for the index corresponding to the word, which is set to one. This method enables simple representation but lacks semantic meaning.



	<i>bear</i>	<i>cat</i>	<i>frog</i>
bear	1	0	0
cat	0	1	0
frog	0	0	1

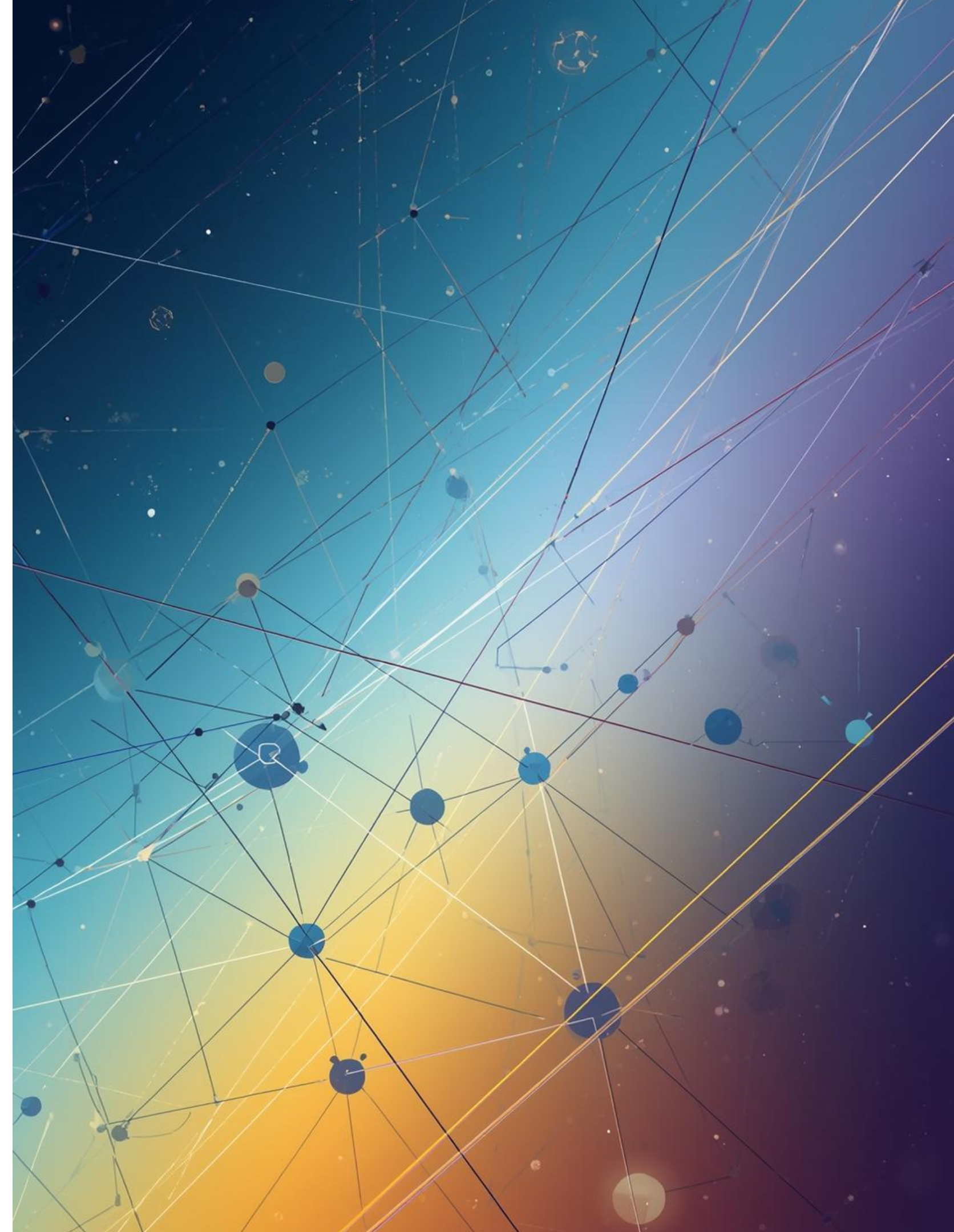
bear = [1,0,0]

cat = [0,1,0]

frog = [0,0,1]

Drawbacks of One-Hot Encoding

- The size of the vector is equal to the number of unique words.
- Leads to sparse vectors.
- Fail to capture the relation between the words.



Count Vectorization

Understanding Word Frequency Representation in Textual Data

Count vectorization transforms text documents into a **numerical format** by representing the frequency of each word in the document. This method allows the mathematical analysis of texts, making it easier to apply algorithms for various Natural Language Processing (NLP) tasks such as classification and clustering.

	FIXTY	TEANC	MIRDTT	PIBOAS	OCUTY	UIIGAS	TEGTT	MAOFT	VIRQJH
1	.501	.803	.025	.871	.824	.003	.019	.003	.007
2	.073	.005	.072	.027	.001	.075	.033	.009	.008
3	.012	.028	.075	.035	.032	.093	.004	.033	.005
4	.505	.033	.033	.003	.022	.034	.004	.098	.093
5	.508	.096	.035	.094	.033	.082	.021	.082	.035
5	.555	.036	.055	.077	.028	.037	.036	.032	.056
4	.505	.000	.034	.024	.083	.099	.034	.033	.038
5	.502	.078	.035	.007	.038	.003	.034	.044	.060
6	.503	.033	.034	.067	.078	.066	.033	.032	.039
6	.507	.034	.093	.037	.028	.057	.037	.069	.031
1	.565	.034	.021	.034	.098	.057	.032	.058	.090
3	.567	.038	.085	.039	.016	.073	.033	.064	.032
5	.553	.034	.065	.057	.035	.025	.024	.040	.029
8	.553	.043	.044	.078	.014	.077	.038	.056	.029
4	.505	.058	.085	.064	.014	.057	.036	.061	.009
0	.555	.034	.035	.033	.014	.035	.036	.089	.016
0	.091	.091	.093	.094	.011	.034	.022	.002	.011
0	.002	.033	.037	.096	.082	.039	.022	.098	.098
1	.001	.078	.036	.097	.096	.062	.025	.068	.069
5	.043	.061	.056	.064	.098	.053	.046	.092	.006

D1: He is a smart boy. She is a smart girl

D2: Rahul is a smart boy

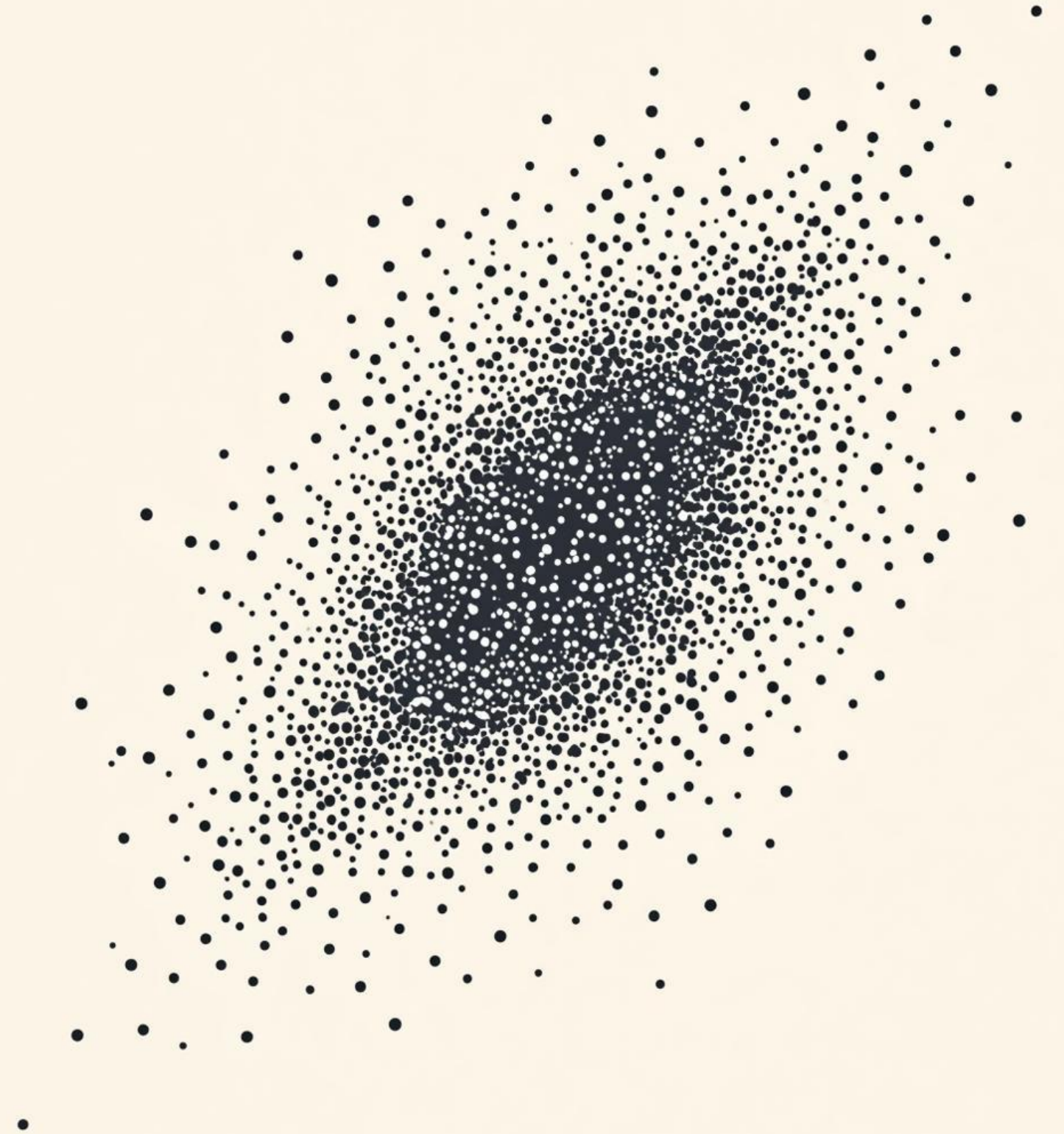
We identify the unique words across all documents, often removing common "stop words."

Dictionary: {he, smart, boy, she, also, rahul}

Document	he	smart	boy	she	also	rahul
D1	1	2	1	1	1	0
D2	0	1	1	0	0	1

Disadvantages of Count Vectorization

- Prone to sparse matrix.
- Fail to capture context information.
- High dimensionality.
- Unable to handle synonyms and polysemy.



Co-occurrence Matrix

A co-occurrence matrix captures how often words appear together within a given context window.

Example Sentence:

“I like NLP and I like AI”

Unique words: [I, like, NLP, and, AI]

Count Co-occurrences:

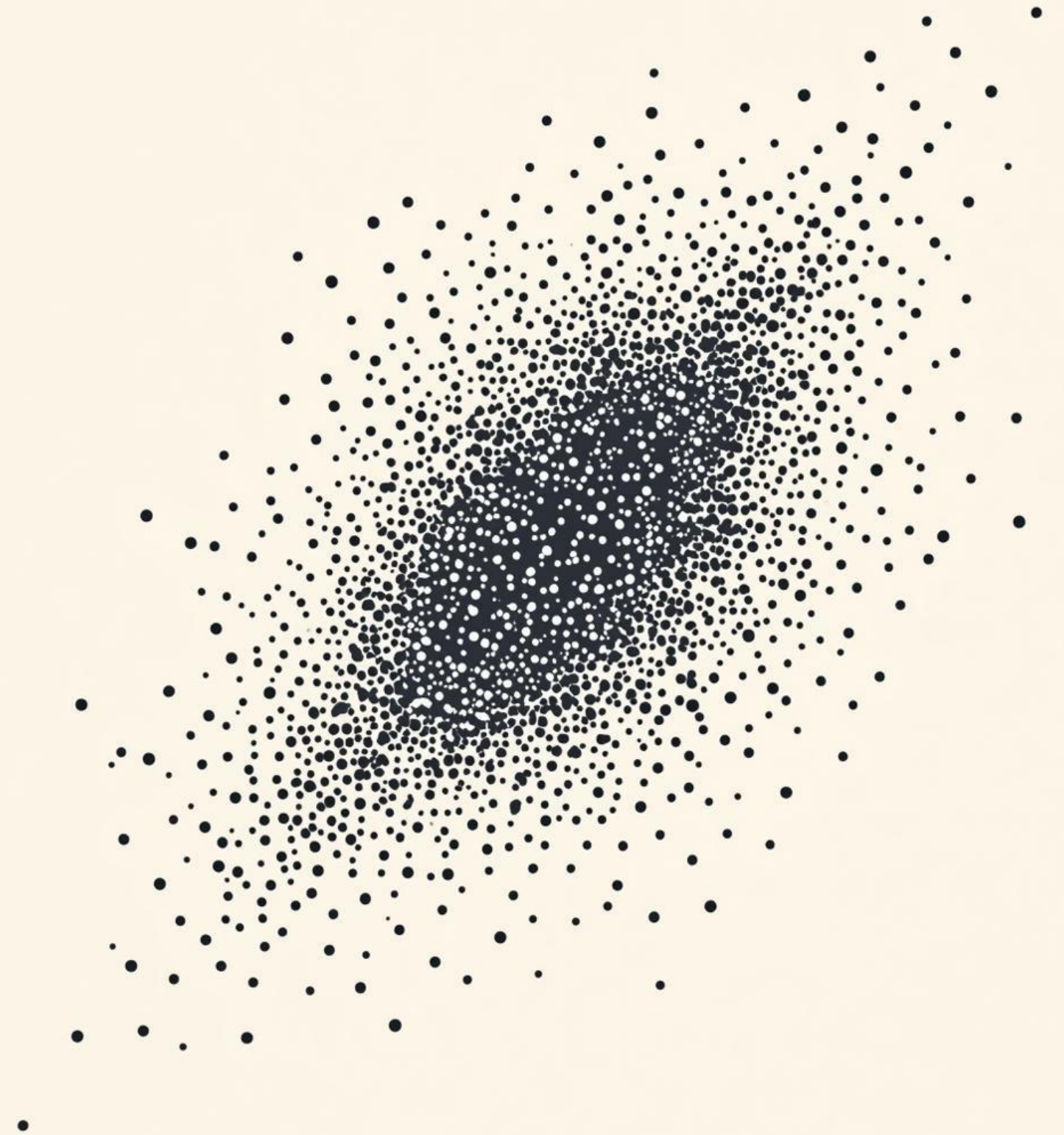
- $I \leftrightarrow \text{like} \rightarrow$ appears twice
- $\text{like} \leftrightarrow \text{NLP} \rightarrow$ once
- $\text{NLP} \leftrightarrow \text{and} \rightarrow$ once
- $\text{and} \leftrightarrow I \rightarrow$ once
- $\text{like} \leftrightarrow \text{AI} \rightarrow$ once

Build the Co-occurrence Matrix

	NLP	and	AI		I	Like
	0	1	0	I	0	2
	1	0	1	Like	2	0
	0	1	0	NLP	0	1
	1	0	0	and	1	0
	0	0	0	AI	0	1

Disadvantages of the co-occurrence matrix

- High dimensionality.
- No semantic understanding.
- Difficulty in selecting the optimal window size.
- Computationally expansive.



TF-IDF

- Previous methods give equal importance to all the words.
- Logic: A word is important if it appears many times in one document (TF), but not if it appears in almost every document in the entire library (IDF).



TF-Term frequency

- Measures how frequently a term occurs in a specific document.
- A word appearing 5 times in a 10-word sentence is more important than appearing 5 times in a 1,000-word essay.

The Formula:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

IDF-Inverse Document Frequency

- Measures how "rare" or "unique" a word is across the entire corpus.
- Words that appear in nearly every document are the least important.
- Important words are that appear frequently in a single corpus and are rare across corpora.

The Formula:

$$IDF(t, D) = \log \left(\frac{\text{Total number of documents } D}{\text{Number of documents containing term } t} \right)$$

Example

D₁: I like NLP

D₂: I like AI

Step 2 : Create vocabulary [I, like, NLP, AI]

Step3 : TF values for D₁ and D₂

Document	I	like	NLP	AI
D1 (3 words)	1/3	1/3	1/3	0
D2 (3 words)	1/3	1/3	0	1/3

Example

Step 4: calculate Inverse Document Frequency(IDF)

$IDF(word) = \log(N / df)$

Where N = 2 (total documents)

I	like	NLP	AI
0	0	0.301	0.301

Example

Step5 : calculate TF*IDF values for both D₁ and D₂

Conclusion:

Words like 'I' and 'like' appear in all documents and hence receive zero weight. Words like 'NLP' and 'AI' are more informative and receive higher TF-IDF scores.

Document	I	like	NLP	AI
D1	0	0	0.100	0
D2	0	0	0	0.100

Assignment

S_1 : good boy

S_2 : good girl

S_3 : boy girl good

Limitations

- No semantic understanding.
- High dimensionality.
- ignore word order: “dog bit man” and “man bit dog” has same meaning.
- Fail to handle synonyms and polysems.

Tokenisation

- Tokenization is the task of chopping text into pieces, called tokens.
- A token is the smallest unit of information.
- Each token is associated with a unique id called “token id”.

Example: to sleep more to learn

Tokens : to, sleep, more, to , learn

Token id : 338, 3123, 278, 1058, 622

How model break down text

Word-based tokenizer:

- Splitting the text by spaces
- Other delimiters such as punctuation can be used

Advantages

- Easy to implement

Disadvantages

- High risk of missing words; e.g., Let and Let's will have two different types
- Languages like Chinese do not have space

How model break down text

Character-based tokenizer:

Approach :

Splitting the text into individual characters

Advantage :

- There will be no or very few unknown words
- useful for languages where characters carry information
- Easy to implement

Disadvantage:

- A character usually does not have meaning.
- fail to capture the semantic information.

How model break down text

Subword tokenizer:

Approach :

- Frequently used words should not be split into smaller subwords.
- Rare words should be decomposed into meaningful subwords.

Advantage :

- Handle unknown words
- Most of the current state-of-the-art models use this method.

Disadvantage:

- more time-consuming than the other two.

How model break down text

Byte-Pair Encoding: used by OpenAI

Uses Huffman encoding for tokenization (greedy algorithm)

Training Steps:

1. Starts with splitting the input words into single characters
(each of them corresponds to a symbol in the final vocabulary)
* In practice we commonly add special end of word symbol “__” before space
2. Find the most frequent occurring pair of symbols from the current vocabulary
3. Add this to the vocabulary and size of vocabulary increases by one
4. Repeat steps (2) and (3) till the defined number of tokens are built
or no new combination of symbols exist with required frequency

How model break down text

Assignment!

How WordPiece is different from BPE?

HuggingFace: <https://huggingface.co/course/chapter6/6?fw=pt>

Paper: <https://arxiv.org/pdf/1609.08144v2.pdf>

Blog: <https://ai.googleblog.com/2021/12/a-fast-wordpiece-tokenization-system.html>

Word2Vec

- Google developed the model in early 2013.

WHY WORD₂VEC:

- Process each word in the vocabulary into numerical vectors.
- Reflect the syntactic and semantic similarity between the words.
- Map multiple words with the same meaning into a single vector.

How Word2Vec different

- It captures the semantic meaning of words: words such as happy and joy are placed in the same vector space.
- Generate low-dimensional vectors: comes 100, 200, and 300 dimensions.
- Generate a dense vector: most of the numbers in the vector are non-zero elements.

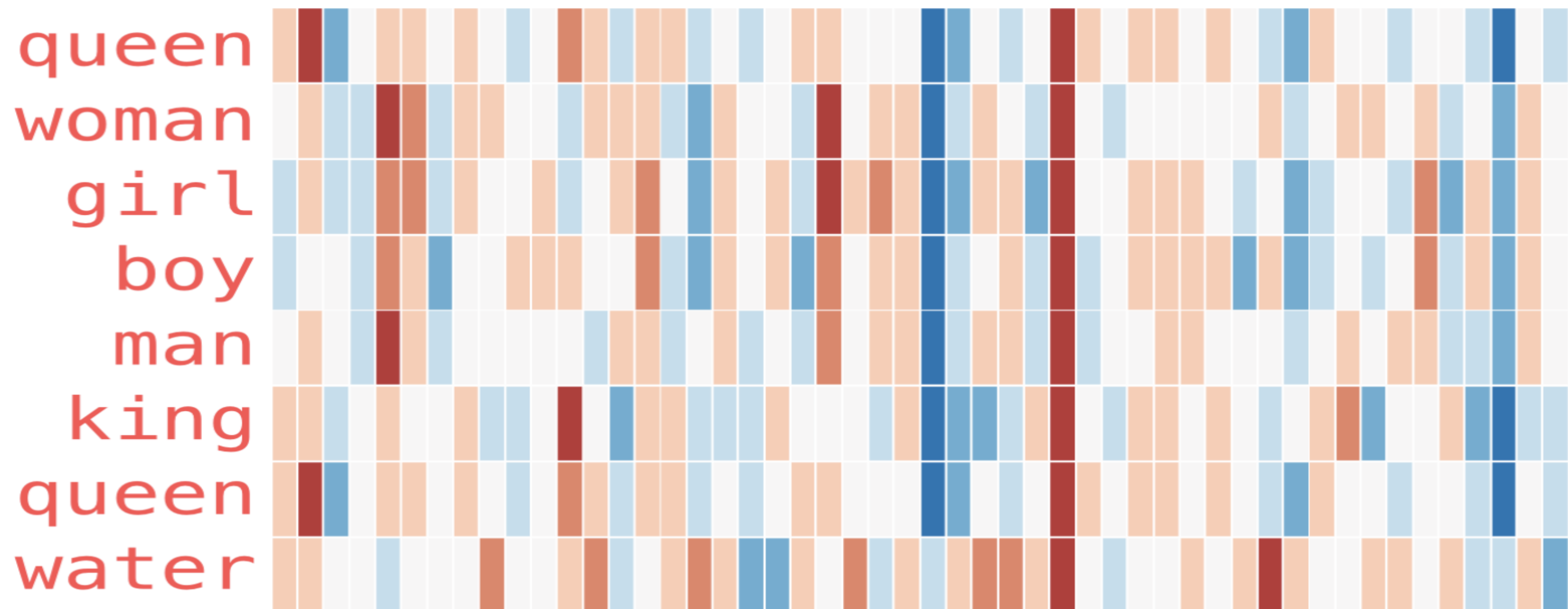
Intuition behind Word2Vec

Queen = King – Man + Woman

	King	Queen	Man	Woman	Monkey
Gender	1	0	1	0	1
Wealth	1	1	0.4	0.4	0
Power	1	1	0.3	0.3	0
Weight	0.8	0.4	0.6	0.4	0.2
Speak	1	1	1	1	0

Intuition behind Word2Vec

Note: Similar context words share similar meaning and vector representation



Word2Vec

- Context word:

$$P(\overset{\checkmark}{w_{k+1}} | w_{i-k}, w_{i-k+1}, \dots, w_k)$$

- ▶ **Continuous Bag of Words (CBOW) Models** – A central word is surrounded by context words. Given the context words, identify the central word
 - ▶ Wish you many more happy returns of the day
- ▶ **Skip Gram Model** – Given the central word, identify the surrounding words
 - ▶ Wish you many more happy returns of the day

CBOW model

- In CBOW, given the context word, we need to predict the target word.
- CBOW uses the middle word as the target word.

MIT	Manipal	Is	Good	College		
	MIT	Manipal	Is	Good	College	
		MIT	Manipal	Is	Good	College

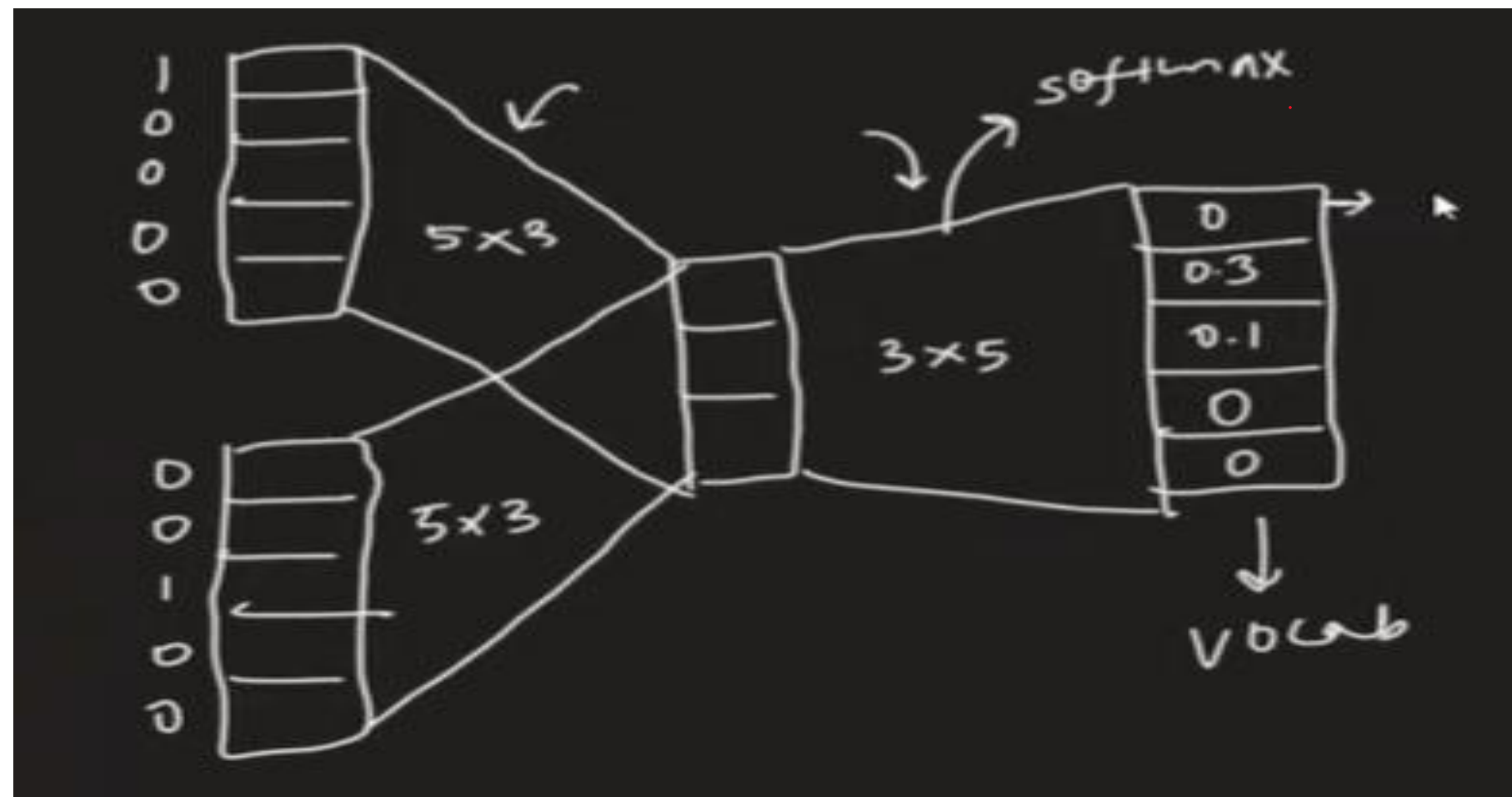
- For a context window of $W=3$. It's a multiclass classification problem with number of classes equal to the vocabulary.

CBOW model

Iteration	Input	Output
1	MIT, Is	Manipal
2	Manipal, Good	is
3	Is, College	Good

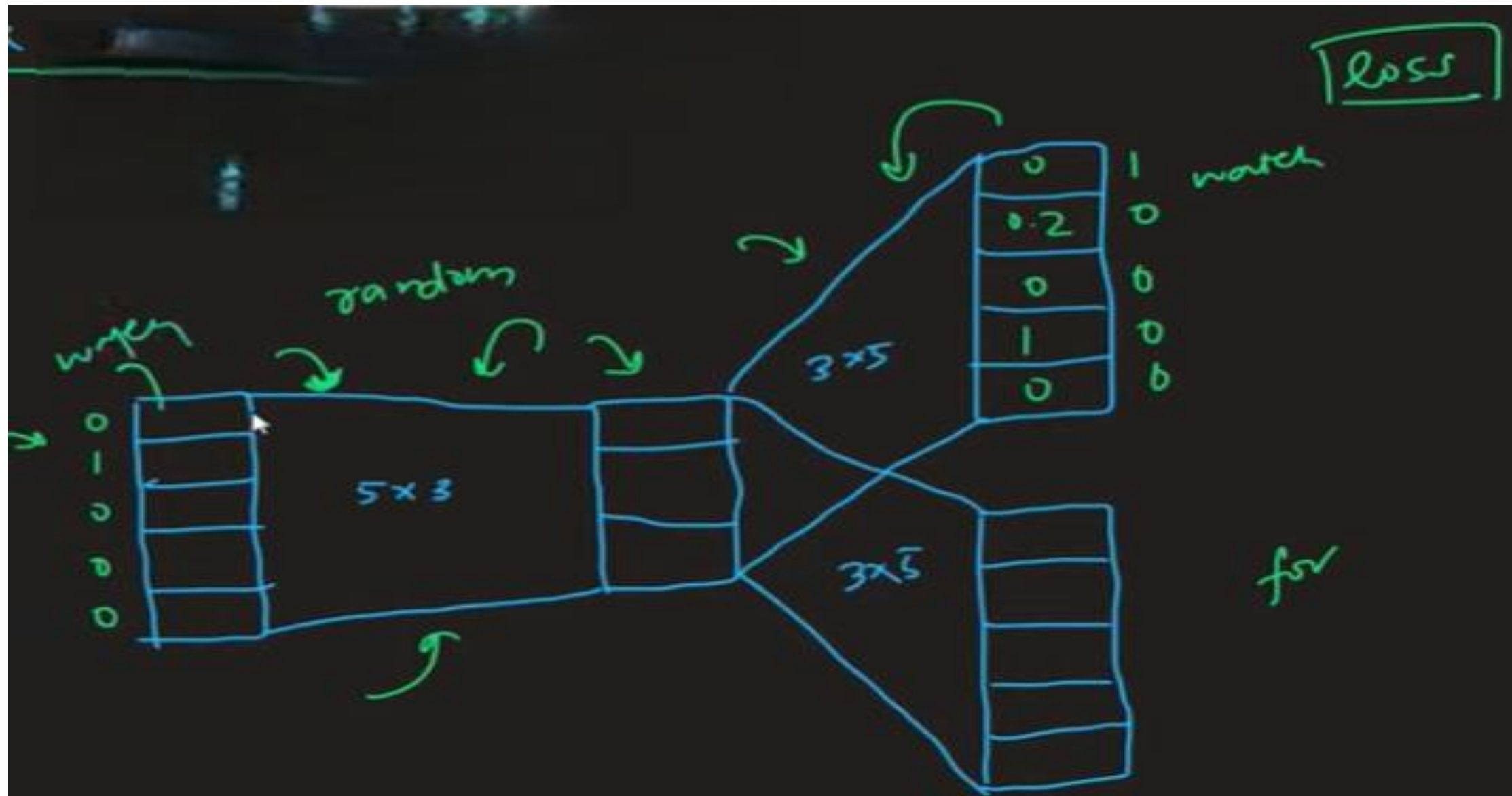
- They built the fully connected neural network model on this training data.
- The input for the model is a one-hot representation.

Architecture of CBOW model



Note: Weights of the last hidden layer in freezed model act as embeddings

Architecture of CBOW model



Note: Weights of the First hidden layer in freezed model act as embeddings

When to use CBOW?

- Large training corpus.
- Frequent words in corpus.
- Rare words are not that important.
- Faster training time.

When to use Skip-gram?

- Smaller training corpus.
- Rare words are important.
- High-quality embeddings are needed.